

Chapter 71: Deployment

Scenario 1: One Build, Many Environments

Situation: Your team builds an Angular 17+ standalone application with the application builder (esbuild-based `@angular/build:application`). The release pipeline currently runs `ng build --configuration=dev`, `ng build --configuration=staging`, and `ng build --configuration=production` separately, producing three different artifacts. QA complains that the artifact that finally reaches production has never actually been tested — it was rebuilt with a different configuration than the one that passed QA in staging. Compliance now requires "build once, promote everywhere."

Question: How do you restructure the app so a single build artifact can be deployed to dev, staging, and production, with environment-specific values (API base URL, feature flags, logging level) injected at deploy time instead of at build time?

Answer: The root cause is using Angular's `fileReplacements` / `environment.ts` mechanism for things that should not be baked into the JS bundle at compile time. `fileReplacements` is a *build-time* mechanism — it necessarily produces a different artifact per environment, which is exactly what compliance is complaining about.

The fix is to externalize runtime configuration into a JSON file that is fetched by the app at bootstrap, and to have the *deployment* step (not the build step) drop the correct `config.json` next to `index.html`.

1. Remove environment-specific secrets/URLs from `environment.ts`. Keep only build-time flags that are truly compile-time (e.g., whether to enable Angular DevTools hooks).
2. Add a `public/config.json` (served as a static asset) with placeholder values:

```
{
  "apiBaseUrl": "https://api.__ENV__.example.com",
  "featureFlags": { "newCheckout": false },
  "logLevel": "warn"
}
```

3. Use an `APP_INITIALIZER` (or the newer `provideAppInitializer`) to fetch it before the app renders:

```
// app.config.ts
import { ApplicationConfig, provideAppInitializer, inject } from '@angular/core';
import { HttpClient, provideHttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';
import { AppConfigService } from './app-config.service';

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideAppInitializer(() => {
      const http = inject(HttpClient);
      const configService = inject(AppConfigService);
      return firstValueFrom(http.get('/config.json')).then(cfg => configService.set(cfg));
    }),
  ],
};
```

```
};
```

4. In the CI pipeline, build exactly once:

```
# .gitlab-ci.yml (excerpt)
build:
  stage: build
  script:
    - npm ci
    - npx ng build --configuration=production
  artifacts:
    paths: [dist/app]

deploy-staging:
  stage: deploy
  script:
    - cp config/staging.config.json dist/app/browser/config.json
    - aws s3 sync dist/app/browser s3://my-app-staging --delete

deploy-prod:
  stage: deploy
  when: manual
  script:
    - cp config/prod.config.json dist/app/browser/config.json
    - aws s3 sync dist/app/browser s3://my-app-prod --delete
```

Now the exact same JS/CSS bundle (same hash, same artifact) is promoted through every stage; only `config.json` — a tiny, cacheable-for-a-short-TTL file — changes per environment. This satisfies "build once, promote everywhere," makes the QA-tested artifact provably identical to the production artifact, and keeps secrets out of the JS bundle (though truly sensitive secrets still shouldn't go in `config.json` either — those belong server-side, behind a proxy the SPA calls).

Tradeoff: this adds one network round trip before bootstrap (mitigate with a small inline critical-CSS loader/splash screen), and you must remember to set `Cache-Control: no-cache` on `config.json` so environments don't get stuck serving stale config from a CDN edge.

Interviewer intent: Tests whether the candidate understands the difference between Angular build-time configuration (`fileReplacements`) and runtime configuration, and whether they can design a genuine "build once, deploy many" pipeline — a very common enterprise/compliance requirement.

Scenario 2: Stale `index.html` After Deploy

Situation: After every production deploy, a subset of users report a blank white screen or `ChunkLoadError` in the console. Investigating, you find that `index.html` was cached by the CDN/browser for an hour, so users kept the old `index.html` (which references `main.a1b2c3.js`), while the new deploy had already deleted that old hashed file from the server, replacing it with `main.d4e5f6.js`.

Question: How do you design the caching strategy so old clients never reference deleted assets, while still getting the benefit of long-lived caching for JS/CSS?

Answer: The Angular CLI already content-hashes output filenames (`outputHashing: "all"` is the default in application-builder production configs), so `main.<hash>.js`, `styles.<hash>.css`, and chunk files are immutable — the same hash always contains the same bytes. The bug is entirely about `index.html` caching policy, not about the hashing itself.

The rule: **hashed files** → **cache forever & immutable**; `index.html` (and any non-hashed root files like `config.json`, `manifest.webmanifest`) → **never cache, or cache very briefly and always revalidate**.

```
# nginx.conf
server {
    listen 80;
    root /usr/share/nginx/html;

    # index.html itself: always revalidate, never let CDN/browser cache it long
    location = /index.html {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
        add_header Pragma "no-cache";
        expires -1;
    }

    # hashed static assets: cache forever, immutable
    location ~* \.(?:js|css)$ {
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    location ~* \.(?:png|jpg|jpeg|gif|svg|woff2?)$ {
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    # SPA fallback (see Scenario 3)
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Additionally, harden the deploy process itself: never delete old hashed assets immediately. Keep at least the previous N releases' hashed files available for a grace window (e.g., 24–48 hours) so any browser tab that already has the old `index.html` loaded — or any CDN edge that hasn't purged yet — can still fetch the old chunks it needs instead of throwing `ChunkLoadError`. Many teams implement this as "deploy to a versioned folder (`/releases/2026-07-23_143000/`), then atomically flip a symlink/CDN origin path," so old and new hashed assets coexist during the transition.

Also add client-side resilience: Angular Router can detect `ChunkLoadError` on lazy-loaded route failures and force a full reload:

```
router.events.subscribe(event => {
    if (event instanceof NavigationError && /Loading chunk .*
failed/.test(event.error?.message ?? '')) {
        window.location.reload();
    }
});
```

Interviewer intent: Checks understanding of cache-busting via content hashing, the specific asymmetry between `index.html` and hashed bundles, and defensive deploy practices (retain old assets, graceful chunk-load-error recovery) — not just "set cache headers."

Scenario 3: SPA Behind an Nginx Reverse Proxy — 404s on Refresh

Situation: The Angular app uses the Router with `provideRouter` and path-based routes like `/dashboard/reports/42`. It's deployed behind an Nginx reverse proxy that also proxies `/api/*` to a backend Node service. Direct navigation to `https://app.example.com/dashboard/reports/42` (or a browser refresh on that URL) returns Nginx's default 404 page, even though clicking through the app to that same URL works fine.

Question: Configure Nginx correctly so deep links and refreshes work, without breaking the `/api` proxy, and explain why the problem happens in the first place.

Answer: The problem happens because Angular's Router does all routing client-side via the History API — `/dashboard/reports/42` is not a real file/directory on the server. When you click a router link, no request goes to the server at all; the Router intercepts navigation and swaps components in-place. But a hard refresh or a typed URL *does* issue a real HTTP GET for `/dashboard/reports/42` to Nginx, which has no such file and no such location block, so it 404s.

The fix is the standard SPA fallback: any request that doesn't match a real static file or an API route should fall back to serving `index.html`, letting Angular's Router take over and resolve the route client-side.

```
server {
    listen 80;
    server_name app.example.com;
    root /usr/share/nginx/html/browser;
    index index.html;

    # 1. API calls go to the backend – must be matched BEFORE the SPA fallback
    location /api/ {
        proxy_pass http://backend-service:3000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 2. Static assets served directly if they exist
    location ~* \.(?:js|css|png|jpg|svg|woff2?|ico)$ {
        try_files $uri =404;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    # 3. Everything else falls back to index.html so Angular Router can handle it
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```

    }

    location = /index.html {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }
}

```

Key points to call out in the interview:

- Location block **ordering/specificity** matters — `/api/` must be matched (and proxied) before the catch-all `/` fallback, otherwise API calls would themselves return `index.html`.
- `try_files $uri $uri/ /index.html` first checks if the literal path exists as a file or directory on disk (so real static assets are still served normally), and only falls back to `index.html` if nothing matches — this correctly returns real 404s for genuinely missing static files while still supporting Angular deep links.
- The server must always respond with HTTP 200 (not 301/404) when serving the fallback `index.html`, otherwise crawlers/CDNs may cache the wrong status.
- If deployed under a sub-path (e.g., `/app/`), you must also set `baseHref / --base-href=/app/` in the Angular build and adjust `root / location` accordingly, or the Router and asset paths will resolve incorrectly.
- For platforms without config-file-level Nginx access (Netlify, Vercel, S3+CloudFront), the equivalent is a `_redirects` file (`/* /index.html 200`) or a CloudFront custom error response mapping 403/404 → `/index.html` with 200.

Interviewer intent: Verifies the candidate understands *why* SPA routing breaks server-side (client-side History API routing vs. real HTTP requests), not just that they can paste a `try_files` line — and whether they can reason about location-block precedence next to a reverse-proxied API.

Scenario 4: Hashed vs. Non-Hashed Asset Caching Policy

Situation: A performance audit shows the app re-downloads all JS/CSS on every visit even though the deploy hasn't changed, wasting bandwidth and hurting repeat-visit load times. Separately, a marketing team complains that after they updated `assets/logo.png` (same filename, new image), users kept seeing the old logo for a week.

Question: Design a comprehensive HTTP caching header strategy for an Angular application builder output that solves both problems simultaneously.

Answer: These are two symptoms of the same missing policy: assets need to be classified by whether their filename changes when their content changes.

Category 1 — Hashed, content-addressed files (`main.<hash>.js`, `styles.<hash>.css`, lazy chunk files, and anything processed through the Angular build pipeline with `outputHashing`): the filename *is* the cache key. If content changes, the filename changes. These can and should be cached forever:

```
Cache-Control: public, max-age=31536000, immutable
```

`immutable` additionally tells supporting browsers not to even revalidate on refresh — a meaningful savings on repeat visits.

Category 2 — Non-hashed files whose name never changes but whose content can (`index.html`, `config.json`, `assets/logo.png`, `manifest.webmanifest`, `favicon.ico` if it's referenced by fixed name): these must never be cached long-term, because there's no way to bust the cache other than the URL itself changing.

```
Cache-Control: no-cache, must-revalidate
```

Note `no-cache` does *not* mean "don't cache" — it means "cache it, but always revalidate with the server (via `ETag / If-None-Match` or `Last-Modified / If-Modified-Since`) before using it." This still saves a full re-download when unchanged (304 Not Modified) while guaranteeing freshness.

The actual fix for the logo problem: either (a) route static assets like logos through the Angular build so they get hashed (put them in `src/assets` and reference via a build-time-resolved path, or better, import them so the bundler fingerprints them), or (b) if they must stay at a fixed path (CMS-managed assets), apply the `no-cache, must-revalidate` policy so at least they're not cached for a long TTL, accepting the cost of revalidation requests.

```
# Category 1: fingerprinted build output
location ~* \.(?:js|css)$ {
    add_header Cache-Control "public, max-age=31536000, immutable";
}

# Category 2: anything else static that isn't fingerprinted
location ~* \.(?:png|jpg|jpeg|gif|svg|ico|webp)$ {
    add_header Cache-Control "public, max-age=86400, must-revalidate";
    # shorter TTL as a middle ground when true hashing isn't feasible
}

location = /index.html {
    add_header Cache-Control "no-cache, must-revalidate";
}
```

Also configure a CDN's edge-cache TTL separately from the browser `Cache-Control` header (e.g., CloudFront `Cache-Policy` / `s-maxage`) since a misconfigured CDN can override or ignore origin headers — and always provide a fast, deploy-triggered CDN invalidation for `index.html` and non-hashed paths as a safety net:

```
- name: Invalidate CDN for non-hashed paths
  run: |
    aws cloudfront create-invalidation \
      --distribution-id $DIST_ID \
      --paths "/index.html" "/config.json" "/assets/logo.png"
```

Interviewer intent: Distinguishes candidates who can recite "cache JS forever" from ones who understand *why* that's only safe when the filename is content-derived, and who can reason about the middle-ground case (non-hashed assets that still change).

Scenario 5: Blue-Green Deployment for a Frontend

Situation: Your company wants zero-downtime deploys and an instant rollback path for the Angular app, similar to what the backend team already does with blue-green deployments for their API. The app is served as static files from behind a load balancer / CDN.

Question: How do you implement blue-green deployment for a purely static Angular SPA, and what Angular-specific pitfalls do you need to guard against (e.g., mixed-version API calls, mismatched chunk hashes)?

Answer: Blue-green for a static SPA maps naturally onto "two complete, independent copies of the release, and an atomic switch of which one traffic points to."

Infrastructure pattern (S3 + CloudFront example):

- Maintain two S3 origins/paths: `s3://my-app/releases/blue/` and `s3://my-app/releases/green/`.
- Deploy the new build to the *idle* color (say, green) while blue continues serving all live traffic.
- Smoke-test green via a private URL or a small percentage of internal traffic.
- Flip the CDN origin (or an ALB target group, or a Route53 weighted/failover record) to point at green.
- Blue is kept warm, untouched, ready for instant rollback (just flip the origin back).

```
# Simplified deploy step
deploy-idle-color:
  script:
    - IDLE_COLOR=$(aws ssm get-parameter --name /app/idle-color --query 'Parameter.Value' --
output text)
    - aws s3 sync dist/app/browser s3://my-app/releases/$IDLE_COLOR --delete
    - echo "Deployed to $IDLE_COLOR, awaiting smoke test"

switch-traffic:
  when: manual
  script:
    - NEW_LIVE=$(aws ssm get-parameter --name /app/idle-color --query 'Parameter.Value' --
output text)
    - aws cloudfront update-distribution --id $DIST_ID --origin-path "/releases/$NEW_LIVE"
    - aws ssm put-parameter --name /app/idle-color --value "$( [ "$NEW_LIVE" = "blue" ] &&
echo green || echo blue )" --overwrite
```

Angular-specific pitfalls:

1. **In-flight sessions during the flip.** A user with an open tab on the "blue" build might have long-lived lazy chunks it hasn't loaded yet. If blue's hashed assets are deleted the moment you switch, that user gets `ChunkLoadError`. Solution: never delete the outgoing color's assets immediately — keep both colors' static assets reachable for a grace period (they're on different paths, so this is cheap and easy in a blue-green setup, unlike single-directory deploys).
2. **API version skew.** If the new frontend build expects new backend API contracts, blue-green on the frontend alone can put an old backend against a new frontend (or vice versa) mid-cutover. Coordinate contract versioning (backward/forward-compatible API changes) or gate the frontend release behind the backend release being fully rolled out first.
3. `index.html` **must switch atomically with everything else**, otherwise you can get a window where `index.html` from green references hashed chunks that don't exist yet on the currently-live color. CDN

origin-path flips are atomic from the client's perspective (one request either hits the old origin path or the new one, never a mix), which is why this pattern is preferred over syncing files in place.

4. **Cache invalidation.** After flipping, invalidate the CDN edge cache for `index.html` specifically (or set very low TTL on it as in Scenario 2) so users see the new build promptly rather than waiting out an edge cache TTL.

Rollback here is close to instant: flip the CDN origin path back to blue — no rebuild, no redeploy, just a routing change, typically propagating in seconds to a couple of minutes depending on CDN.

Interviewer intent: Tests whether the candidate can translate a backend deployment pattern (blue-green) to a static-asset frontend context, and whether they think about the SPA-specific failure mode of in-flight clients referencing assets that get deleted mid-cutover.

Scenario 6: Canary Releasing a New Frontend Build

Situation: Leadership wants to de-risk a major Angular version upgrade (with a redesigned checkout flow) by exposing it to only 5% of production traffic first, watching error rates and conversion metrics, then ramping to 25%, 50%, 100% — or rolling back instantly if metrics regress.

Question: Design a canary deployment strategy for the SPA. Where does the traffic-splitting decision get made, and how do you avoid inconsistent experiences for a single user across page loads?

Answer: There are two common places to implement the split, each with different tradeoffs:

Option A — Edge/CDN-level splitting (recommended for SPAs). Deploy the canary build to a separate path/origin (like blue-green) and use a CDN feature (CloudFront + Lambda@Edge/CloudFront Functions, Fastly VCL, or a load balancer's weighted routing) to route a percentage of requests for `index.html` to the canary origin. Critically, pin the decision to a **stable cookie**, not a fresh random roll per request, so a given user consistently gets the same version across their session (otherwise a user could get canary's `index.html` but stable's cached chunk, or bounce between two different UIs on every navigation — very confusing, and it corrupts your metrics because you can't attribute a full session to one variant).

```
// CloudFront Function (viewer request) – simplified
function handler(event) {
  var request = event.request;
  var cookies = request.cookies;
  var variant;

  if (cookies['app-variant']) {
    variant = cookies['app-variant'].value;
  } else {
    variant = Math.random() < 0.05 ? 'canary' : 'stable';
    // set-cookie handled by an accompanying response function / origin
  }

  request.uri = variant === 'canary'
    ? '/releases/canary' + request.uri
    : '/releases/stable' + request.uri;

  return request;
}
```

Option B — Application-level feature flagging. Ship one build containing both old and new checkout flows behind a feature flag service (LaunchDarkly, Unleash, or a simple in-house flag endpoint), and use `@if /` structural directives to render one or the other based on the flag evaluated per-user at runtime.

```
@Component({
  template: `
    @if (flags.newCheckout()) {
      <app-checkout-v2 />
    } @else {
      <app-checkout-v1 />
    }
  `,
})
export class CheckoutPageComponent {
  flags = inject(FeatureFlagService);
}
```

Option B is often *preferred* for Angular apps specifically because it avoids the "two full copies of static assets, atomic edge routing" infrastructure complexity of Option A, ships a single build (satisfying the "build once" principle from Scenario 1), and lets you target by user attributes (beta cohort, account tier) rather than just a random bucket — at the cost of extra bundle size (both code paths ship) and needing to eventually clean up the flag and delete the old code path (flag debt).

For a *major version upgrade* specifically (the scenario given), Option A is usually necessary anyway, because the two versions may not even share a build (different Angular major, different bundler output) — you can't cleanly feature-flag between "Angular 15 build" and "Angular 20 build" inside one bundle. So: canary at the infra/CDN level for major version/framework upgrades; feature-flag at the app level for incremental feature rollouts within the same build.

Rollback: for Option A, simply set the CDN split back to 0% canary / 100% stable — no rebuild needed. For Option B, flip the flag off (or kill-switch it) instantly server-side without any frontend redeploy at all, which is the single biggest advantage of feature-flag-based canaries.

Interviewer intent: Probes whether the candidate distinguishes infrastructure-level canary (separate builds, edge routing, sticky sessions) from application-level canary (feature flags in one build), and understands when each is appropriate — especially the "two builds can't share a bundle across a major version jump" insight.

Scenario 7: Emergency Rollback of a Bad Deploy

Situation: Fifteen minutes after a production deploy, error-tracking (Sentry) shows a 40x spike in uncaught exceptions on the checkout page. The on-call engineer needs to get the site back to a known-good state in under two minutes, not by debugging the new code but by reverting.

Question: Walk through exactly how you'd roll back an Angular SPA deployment quickly and safely, and what pre-deploy practices make this possible in an emergency versus scrambling.

Answer: The speed of rollback is entirely determined by decisions made *before* the incident, not during it. Three prerequisites make a sub-two-minute rollback possible:

1. **Immutable, versioned releases** — every deploy goes to its own path/tag, never overwriting a shared "live" directory in place:

```
s3://my-app/releases/2026-07-23_143000-a1b2c3d/  
s3://my-app/releases/2026-07-22_091500-9f8e7d6/    <- previous, known-good
```

2. **A single atomic pointer** that decides what's "live" — a CDN origin-path, a symlink, or a small routing rule — never a fragile in-place file sync.

3. **The previous release's artifacts are retained and untouched**, so rollback needs zero rebuild.

Given that, the rollback itself is one command:

```
# CloudFront example - flip origin path back to the last known-good release  
aws cloudfront get-distribution-config --id $DIST_ID > /tmp/current.json  
# ... update OriginPath from /releases/2026-07-23_143000-a1b2c3d  
#                               to /releases/2026-07-22_091500-9f8e7d6  
aws cloudfront update-distribution --id $DIST_ID --distribution-config  
file:///tmp/updated.json  
aws cloudfront create-invalidation --distribution-id $DIST_ID --paths "/index.html"
```

Or, if using Kubernetes-hosted Nginx serving the static files from an image:

```
kubectl rollout undo deployment/frontend --to-revision=<previous>  
kubectl rollout status deployment/frontend
```

Or, for Vercel/Netlify-style platforms, it's a one-click/one-CLI-call "promote previous deployment" action:

```
vercel rollback <previous-deployment-url>
```

Also important — communicate the rollback distinctly from a redeploy: rollback should *not* re-trigger the full CI build pipeline (that reintroduces build-time risk and burns minutes you don't have during an incident); it should just re-point traffic to artifacts that already passed CI and were already live once.

Post-rollback follow-ups to mention: invalidate `index.html` /non-hashed paths in the CDN (Scenario 2) so clients pick up the reverted `index.html` promptly instead of waiting on TTL; keep the bad release's artifacts around (don't delete) for forensics; and if the bug was already server-side-persisted (e.g., a schema migration ran with the bad deploy), a frontend-only rollback isn't sufficient — coordinate with backend rollback too, since frontend/backend contract mismatch can itself cause the very same kind of error spike.

Finally, mention *feature flags as a faster-than-rollback option*: if the broken checkout flow was behind a flag (as discussed in Scenario 6), flipping the flag off is even faster than a CDN pointer flip and doesn't require the "keep every past release available" infrastructure at all.

Interviewer intent: Tests operational maturity — whether the candidate designs deployments to be *reversible by construction* (immutable releases + atomic pointer) rather than treating rollback as an afterthought that requires reverting code and rebuilding under pressure.

Scenario 8: Deploying Angular SSR to a Container Platform

Situation: You've enabled Angular SSR (`ng add @angular/ssr`) for better SEO and faster First Contentful Paint. The app now ships a Node Express server (`server.ts`) alongside the browser bundle. You need to deploy this to a container platform (e.g., Google Cloud Run / AWS ECS / Kubernetes) with autoscaling, and separately someone asks whether it could instead run on a serverless platform.

Question: What does the build output look like for SSR, how do you containerize it correctly, and what are the operational differences (scaling, cold starts, cost) versus a serverless deployment?

Answer: With the application builder, `ng build` with SSR enabled produces two output trees:

```
dist/app/
  browser/      <- static client bundle (served to the browser, same as a pure SPA)
  server/
    server.mjs  <- Node entry point (Express-based request handler)
```

Containerized (Cloud Run / ECS / K8s) deployment:

```
# Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx ng build

FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/dist/app ./dist/app
COPY --from=build /app/node_modules ./node_modules
EXPOSE 4000
CMD ["node", "dist/app/server/server.mjs"]
```

```
# Cloud Run service (excerpt), or equivalent ECS task def / k8s Deployment
service: frontend-ssr
runtime: container
port: 4000
resources:
  cpu: "1"
  memory: 512Mi
scaling:
  minInstances: 1      # avoid cold starts for a user-facing SSR endpoint
  maxInstances: 20
env:
  - name: API_BASE_URL
    value: "https://api.example.com"
```

Key operational points:

- `minInstances: 1` (or equivalent "always-warm" setting) matters a lot for SSR specifically, because a cold Node process has to boot the Angular server-side render pipeline (module graph, Zone-less hydration setup, etc.) before it can respond — a cold request can be materially slower than a warm one, hurting exactly the FCP metric SSR was adopted to improve.
- Runtime config (Scenario 1's pattern) still applies but is now easier — it can be plain environment variables read by `server.ts` at request time, no `config.json` fetch needed, since the server process can read `process.env` directly per request/at boot.
- Health checks should hit a lightweight endpoint, not a full SSR render of a heavy page, to keep autoscaler decisions fast and cheap.
- CDN/edge caching of SSR'd HTML is trickier than a pure SPA's static `index.html` — full-page HTML now varies per route and potentially per user, so you typically cache at a shorter TTL or bypass cache for authenticated/personalized routes, while still caching the static `browser/` assets aggressively as in Scenario 4.

Serverless (e.g., AWS Lambda / Cloud Functions / Vercel Edge) alternative: Angular's SSR server can be adapted to a serverless handler (Angular's Node engine can be wrapped with something like `@vendia/serverless-express` or platform-specific adapters, and some hosts like Vercel/Netlify have first-class Angular SSR support that does this automatically).

Tradeoffs vs. containers:

- **Cost model:** serverless bills per-invocation/duration, ideal for spiky/low/unpredictable traffic; a min-instance-1 container bills continuously even at zero traffic, better for steady, high-volume traffic where cold starts are unacceptable.
- **Cold starts:** serverless cold starts can be worse for SSR than for a simple API function, because the SSR bundle (Angular's platform-server renderer, zone.js if used, component graph) is heavier to initialize than a typical small Lambda; mitigate with provisioned concurrency (Lambda) or "keep warm" pings, at which point much of the cost advantage over containers erodes.
- **Execution time limits:** serverless platforms impose hard timeouts (e.g., Lambda's max) — fine for typical SSR renders (tens to low hundreds of ms) but risky if a slow downstream API call blocks the render.
- **Statelessness:** both models require the SSR server to be fully stateless (no in-memory session/cache that assumes the same instance handles the next request), since both autoscale by spinning up/down instances or invoking fresh function instances.

Recommendation given in the answer: for steady, latency-sensitive, high-traffic production apps, containers with `minInstances >= 1` and horizontal autoscaling are usually the safer, more predictable-latency choice; serverless is attractive for low-traffic apps, preview/staging environments, or platforms (Vercel) that have already solved the cold-start problem for you at the platform level.

Interviewer intent: Confirms the candidate actually understands the SSR build output shape (dual browser/server trees) and can reason concretely about container vs. serverless operational tradeoffs specific to a stateful-feeling-but-must-be-stateless SSR Node process, not just "SSR needs Node."

Scenario 9: Stale Service Worker Serving an Old App After Deploy

Situation: The app uses `@angular/service-worker` (PWA) for offline support. After deploying a critical bug fix, several users report they're still seeing the buggy version even after refreshing, force-refreshing, and clearing their browser cache in some cases. Some users are stuck on a version that's three deploys old.

Question: Explain why this happens given how Angular's service worker update model works, and how do you both fix the immediate stuck users and prevent this class of problem going forward?

Answer: This is the classic Angular service worker "update lifecycle" gotcha. `ngsw-worker.js` deliberately does **not** serve new content to an already-open tab immediately — by design, a currently active service worker keeps serving the app it already cached (`ngsw.json` manifest + hashed assets) from cache-first, and only checks for a new version in the background. A regular refresh (even Ctrl+F5 in some cases) can still be answered by the *already-installed* service worker before it's swapped, because the SW controls the page and intercepts the navigation request itself — a normal cache-clear doesn't necessarily unregister/bypass an active service worker, which is a separate browser storage partition (Cache Storage / IndexedDB-backed) from the disk HTTP cache.

By default, the new SW version is downloaded and installed in the background, but doesn't *activate* and take control until all tabs of the old version are closed (this is the standard Workbox/SW lifecycle: `waiting` → `activated` only after `skipwaiting` / client takeover or all clients close) — unless the app explicitly opts into prompting/forcing the update.

Fix for stuck users (immediate): they need to either close all tabs of the app (letting the new SW activate naturally) or the app needs to explicitly call `swUpdate` to force it:

```
// app.config.ts / a core service, initialized at bootstrap
import { inject } from '@angular/core';
import { SwUpdate, VersionReadyEvent } from '@angular/service-worker';
import { filter } from 'rxjs/operators';

export class AppUpdateService {
  private swUpdate = inject(SwUpdate);

  constructor() {
    if (this.swUpdate.isEnabled) {
      this.swUpdate.versionUpdates
        .pipe(filter((evt): evt is VersionReadyEvent => evt.type === 'VERSION_READY'))
        .subscribe(() => {
          // Prompt or auto-reload
          if (confirm('A new version of this app is available. Load it now?')) {
            document.location.reload();
          }
        });

      // Also actively check on load, don't just wait for the SW's own poll interval
      this.swUpdate.checkForUpdate();
    }
  }
}
```

For genuinely stuck/unresponsive clients (e.g., they dismissed the prompt and never revisit), a more forceful pattern is to auto-reload without prompting for critical fixes, or to version-gate API calls so the old client is rejected server-side and forced to reload:

```
this.swUpdate.versionUpdates
  .pipe(filter((e): e is VersionReadyEvent => e.type === 'VERSION_READY'))
  .subscribe(() => document.location.reload());
```

Prevention going forward:

1. Always wire up `swUpdate.versionUpdates` — never ship the SW without an update-detection/reload path; this is arguably a required piece of any `@angular/service-worker` integration, not optional polish.
2. Set `registrationStrategy` appropriately (e.g., `registerWhenStable: 30000` so the SW registers after the app is stable, not instantly competing with initial render) and consider explicitly calling `checkForUpdate()` on an interval or on `visibilitychange` (tab refocus) so long-lived open tabs notice updates rather than relying solely on the browser's own (much longer, ~24h) SW update check interval.
3. For a genuinely critical hotfix, consider an **emergency kill-switch**: an unhashed, no-cache endpoint the app polls (or a push via `BroadcastChannel` /websocket) that says "minimum required version is X," and if the running app is below that, force `navigator.serviceworker.getRegistration().then(reg => reg.unregister()).then(() => location.reload())`.
4. Ensure the `ngsw-config.json` `index` and asset groups are configured so `index.html` is always in the `prefetch` /checked-first group, and understand that the SW's own manifest hash comparison is what detects "there's a new version," so a broken deploy that doesn't change hashes (e.g., you only changed `config.json`'s content — see Scenario 1) won't even be seen as an update by the SW; runtime config changes need their own cache-busting/version query param independent of the SW.

Interviewer intent: Distinguishes candidates who've actually operated a `@angular/service-worker` PWA in production (and hit this exact "why won't it update" support ticket) from those who only know the tutorial-level "add PWA support" step; tests understanding of SW lifecycle (install/waiting/activate) and Angular's `swUpdate` API specifically.

Scenario 10: Cross-Origin API Calls Failing Only in Production

Situation: Locally, the Angular dev server proxies `/api` calls to a backend via `proxy.conf.json` and everything works. After deploying the built app to production (served from `https://app.example.com`, API at `https://api.example.com`), all API calls fail with CORS errors in the browser console, despite the backend team insisting "CORS is enabled."

Question: Diagnose the likely causes and describe the two architecturally different fixes (reverse-proxy same-origin vs. correct CORS headers), including their tradeoffs.

Answer: The dev-server proxy silently hid an architectural decision: in dev, both "the app" and "the API" appear same-origin to the browser (because `ng serve`'s proxy makes `/api/*` requests actually originate from `localhost:4200` itself, not cross-origin). In production, with the app and API on genuinely different origins/domains, the browser enforces CORS for real, and any mismatch (missing `Access-Control-Allow-`

`origin` for that exact origin, missing headers for preflight `OPTIONS`, or credentials mode mismatches) surfaces as an outright blocked request — often confusingly reported as a generic network failure, not a clear "CORS" message, if the preflight itself 404s or times out.

Common root causes to check:

- Backend's CORS allow-list has `https://app-staging.example.com` but not `https://app.example.com` (or vice versa) — environment-specific misconfiguration.
- Preflight (`OPTIONS`) requests aren't handled by the backend at all (common when CORS middleware is applied only to some routes/methods).
- The frontend sends `withCredentials: true` (cookies) but the backend responds with `Access-Control-Allow-Origin: *`, which is invalid combined with credentials — must be an explicit origin.

Fix A — Reverse-proxy to same-origin (recommended for most SPA+API pairs): Put Nginx (or the CDN) in front of both the static app and the API under one origin, eliminating CORS entirely rather than configuring around it:

```
server {
    listen 443 ssl;
    server_name app.example.com;

    location /api/ {
        proxy_pass https://internal-api-service:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        root /usr/share/nginx/html/browser;
        try_files $uri $uri/ /index.html;
    }
}
```

Now the browser only ever talks to `app.example.com` — no CORS negotiation happens at all, no preflight overhead, and cookies (session auth) work without any `SameSite`/CORS credential gymnastics. Tradeoff: couples the frontend's edge infrastructure to also route API traffic, and adds one hop of latency/another moving part to the static-hosting layer (though this is usually negligible and often desirable anyway for things like Scenario 3's routing).

Fix B — Proper CORS configuration on the backend, when the API is genuinely meant to be called from multiple different frontend origins (e.g., a public API also consumed by third parties) and same-origin proxying isn't appropriate:

```
// Express example
app.use(cors({
  origin: ['https://app.example.com', 'https://app-staging.example.com'],
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
```

Here the tradeoff is operational: every new frontend environment (a new preview/staging URL, a new subdomain) requires a backend config change and redeploy to add it to the allow-list — a maintenance burden Fix A avoids entirely.

Either way, the interview-worthy insight is: **the dev-server proxy (`proxy.conf.json`) is a development convenience that hides a same-origin decision you still have to make explicitly for production** — it's not itself a deployment strategy, and teams that don't realize this get blindsided exactly as in this scenario.

Interviewer intent: Checks whether the candidate understands that `ng serve`'s proxy is dev-only tooling, and can articulate the real architectural choice (same-origin reverse proxy vs. cross-origin CORS) a production deployment has to make.

Scenario 11: Multi-Tenant App Deployed to a Single CDN with Per-Tenant Theming

Situation: The Angular app serves multiple corporate tenants (`tenantA.example.com` , `tenantB.example.com`) from a single codebase, with each tenant needing different logos, color themes, and a couple of feature toggles, but otherwise identical functionality. Product wants new tenants to be onboardable "in an afternoon" without a code change or rebuild.

Question: How would you architect the build/deploy so that a single Angular build artifact serves all tenants with per-tenant branding resolved at runtime, and how does the CDN/hosting layer need to route requests?

Answer: This combines the runtime-config pattern from Scenario 1 with host-based routing at the CDN/reverse-proxy layer, and is a good test of whether the candidate over-engineers with per-tenant *builds* (which fails the "afternoon onboarding, no rebuild" requirement) versus a single build with tenant resolved at request time.

1. **One build.** Never use `fileReplacements` per tenant — that's exactly the anti-pattern from Scenario 1, multiplied by N tenants, and guarantees onboarding a new tenant requires a code change and rebuild.
2. **Tenant resolution at runtime**, keyed off `window.location.hostname`:

```
// tenant-config.service.ts
@Injectable({ providedIn: 'root' })
export class TenantConfigService {
  private http = inject(HttpClient);

  async load(): Promise<TenantConfig> {
    const host = window.location.hostname; // e.g. "tenantA.example.com"
    return firstValueFrom(this.http.get<TenantConfig>(`/tenant-config/${host}.json`));
  }
}
```

3. **Tenant config files live outside the app bundle**, in a separate, independently-updatable store (an S3 prefix, a small config API, or even a CMS) — this is the artifact that changes when onboarding a tenant, not the Angular build:

```
// tenant-config/tenanta.example.com.json
{
  "displayName": "Tenant A Corp",
  "logoUrl": "https://cdn.example.com/tenants/tenanta/logo.svg",
  "theme": { "primary": "#0b5fff", "secondary": "#003a99" },
  "features": { "billingModule": true }
}
```

4. **CDN/Nginx routes all tenant hostnames to the same static app**, since it's the same bundle for everyone:

```
server {
  listen 443 ssl;
  server_name ~^(?<tenant>.+)\.example\.com$; # matches any tenant subdomain

  ssl_certificate      /etc/ssl/wildcard.example.com.crt;
  ssl_certificate_key  /etc/ssl/wildcard.example.com.key;

  root /usr/share/nginx/html/browser; # identical for every tenant
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

A wildcard TLS cert (`*.example.com`) or an automated cert-per-subdomain (e.g., via a cert-manager/ACM with DNS validation) handles new tenant subdomains without manual cert work.

5. **CSS theming applied at runtime** via CSS custom properties set from the loaded tenant config, rather than per-tenant compiled stylesheets:

```
document.documentElement.style.setProperty('--color-primary', tenantConfig.theme.primary);
```

Onboarding a new tenant then becomes: add a DNS CNAME/subdomain, drop one new JSON config file, done — genuinely an "afternoon" task with zero Angular rebuild, matching the requirement. The tradeoff versus per-tenant builds: you lose the ability to have tenants on genuinely different app *versions* (all tenants are always on whatever the single build's current deployed version is) and any tenant-specific feature that needs materially different compiled code (not just config/theme) still needs a feature flag inside the shared bundle (bundle-size cost) rather than a clean separate build.

Interviewer intent: Tests whether the candidate can apply the runtime-vs-build-time configuration principle to a realistic multi-tenant SaaS deployment scenario, and whether they reach for host-based routing + runtime theming instead of the tempting-but-wrong "just build N times" shortcut.

Scenario 12: `base href` Misconfiguration Breaks Deep Links Under a Sub-Path

Situation: The Angular app is deployed not at a domain root but under a sub-path, `https://intranet.example.com/tools/reporting/`, alongside several other unrelated tools hosted under `/tools/*`. After deployment, the app loads fine at the exact root URL, but all internal navigation results in 404s, and all asset requests (`main.js`, `styles.css`) 404 too because the browser is requesting them from `/main.js` instead of `/tools/reporting/main.js`.

Question: What's misconfigured, and what's the complete fix across the Angular build, the router, and the web server?

Answer: The browser is resolving relative asset URLs against the wrong base because `index.html`'s `<base href="/">` (the Angular CLI's default) tells both the browser (for relative asset resolution) and Angular's Router (`PathLocationStrategy`) that the app's root is `/`, when it's actually `/tools/reporting/`.

Fix 1 — set the correct base href at build time:

```
ng build --base-href=/tools/reporting/
```

This rewrites `<base href="/tools/reporting/">` in the generated `index.html`, which fixes both:

- Relative asset resolution (`main.js` becomes requested as `/tools/reporting/main.js`, since the browser resolves relative URLs against `<base href>`).
- Angular Router's internal path construction (routes now generate/parse as `/tools/reporting/dashboard` rather than `/dashboard`).

Fix 2 — Nginx must serve and fall back correctly under that sub-path:

```
location /tools/reporting/ {  
    alias /usr/share/nginx/html/reporting-app/browser/;  
    try_files $uri $uri/ /tools/reporting/index.html;  
}
```

Note the `alias` (not `root`) usage here — with `alias`, the matched location prefix (`/tools/reporting/`) is stripped and replaced entirely by the alias path, whereas `root` would append the full request path onto the root, doubling the sub-path. This `alias` vs `root` distinction is a classic Nginx gotcha specifically for sub-path-deployed SPAs.

Fix 3 — if the deep-link problem persists even with correct `base-href`, check that any hardcoded absolute paths in the app (e.g., `` instead of ``, or an HTTP interceptor building URLs from `/api/...` assuming root) are changed to be relative or explicitly prefixed — an absolute leading-slash path always resolves from the domain root regardless of `<base href>`, silently bypassing the sub-path.

Fix 4 — Router configuration rarely needs explicit changes beyond `base-href` since `PathLocationStrategy` reads `<base href>` automatically, but if `APP_BASE_HREF` is provided manually (e.g., in SSR contexts where there's no real `<base>` tag/DOM), it must match:

```
{ provide: APP_BASE_HREF, useValue: '/tools/reporting/' }
```

Interviewer intent: Tests a specific, easy-to-overlook deployment detail (sub-path hosting) that trips up teams deploying multiple apps under one domain, and checks whether the candidate knows the `alias` vs `root` Nginx distinction, not just the Angular-side `--base-href` flag.

Scenario 13: Choosing Between Docker Multi-Stage Build and Static Hosting

Situation: A new team member proposes containerizing the Angular SPA as an Nginx image built via a multi-stage Dockerfile, arguing "everything should be a container for consistency with our Kubernetes-based backend." Another engineer argues a pure static SPA has no business running in a container at all and should go straight to S3+CloudFront or equivalent object storage/CDN hosting.

Question: Evaluate both positions. When is containerizing a static Angular build the right call, and when is it wasteful?

Answer: Both engineers are half-right; the correct answer depends on organizational context, not on a universal "SPAs should never/always be containers" rule.

Case for object-storage + CDN (S3/CloudFront, Azure Static Web Apps, Cloudflare Pages, Netlify, Vercel):

- A pure static SPA has no server-side logic to run — an Nginx container serving static files is strictly *more* infrastructure than necessary: you're now responsible for patching the Nginx image's OS/CVEs, managing container autoscaling for what is fundamentally a "serve files from disk" job, and paying for always-on compute (even minimal) versus pay-per-request/near-zero-idle-cost object storage.
- CDNs natively provide edge caching, global distribution, and automatic HTTPS — an Nginx-in-a-container setup would need its own CDN in front of it anyway to get equivalent global latency, at which point the container added a layer with no benefit.
- Deploy speed: `aws s3 sync` + CDN invalidation is typically faster than building and rolling out a new container image.

Case for containerizing the static build anyway:

- **Organizational consistency** is a legitimate reason, not just a soundbite: if the platform team's tooling, secrets management, network policies, observability (log/metric shipping), and deployment pipelines are all built around Kubernetes, forcing the frontend team to learn/operate an entirely separate deployment stack (S3 IAM policies, CloudFront distributions, separate CI stages) has a real cost too — sometimes higher than "one more Nginx container."
- If the app is SSR (Scenario 8), containers (or serverless) are **required** anyway, since there's a real Node process to run — so if the org is heading toward SSR, standardizing on containers now avoids a later migration.
- Air-gapped/on-prem/regulated environments without access to public cloud object storage/CDN services sometimes have no practical alternative to container-based hosting.
- Local dev/prod parity: a containerized Nginx setup lets developers run the *exact* production serving configuration (same `nginx.conf`, same headers) locally via `docker run`, which is harder to replicate exactly with a CDN's proprietary edge configuration.

Recommendation to give in an interview: default to static hosting (S3/CDN or a platform like Netlify/Vercel/Cloudflare Pages) for a pure SPA when there's no organizational reason to do otherwise — it's simpler, cheaper, and has fewer moving parts to secure/patch. Containerize when (a) SSR is involved, (b) the org's existing deployment/observability tooling is Kubernetes-centric and adding a second paradigm has a real cost, or (c) environment constraints (on-prem/air-gapped) require it. A reasonable multi-stage Dockerfile for the container option, for completeness:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx ng build

FROM nginx:1.27-alpine
COPY --from=build /app/dist/app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Interviewer intent: Looks for pragmatic, context-driven architectural judgment rather than dogma in either direction — a strong signal of senior-level thinking about deployment infrastructure tradeoffs.

Scenario 14: Feature-Flag Coupled Deploy Causes a Broken Intermediate State

Situation: A deploy ships both a new Angular component (`NewPricingCalculator`) and removes an old API endpoint (`/api/v1/pricing`) it used to call, replacing it with `/api/v2/pricing`. The backend and frontend are deployed by separate pipelines. The backend deploy finishes five minutes before the frontend one during a routine release, and during that window some users on the *old, cached* frontend (from before the deploy started propagating) call the now-removed `/api/v1/pricing` and get 404s, spiking error alerts.

Question: How should deployment ordering and API versioning be handled so frontend/backend deploys of a decoupled SPA+API architecture don't break each other during the rollout window, even though they're on independent pipelines?

Answer: This is a distributed-rollout/versioning problem, not really an Angular-specific one, but it's very commonly tested because SPA+API decoupling makes it uniquely easy to get wrong — unlike a monolith where frontend and backend deploy atomically together, an SPA and its API are almost always on independent deploy cadences, and *any number of old and new clients can be talking to any number of old and new API versions simultaneously* during a rollout, plus for an indefinite period afterward (cached `index.html`, service worker lag from Scenario 9, users who simply haven't refreshed).

The core principle: **never deploy a breaking backend change and a frontend change that depends on it as if they were atomic — they aren't, and won't be, given independent pipelines and client-side caching.** Instead, sequence deploys so there is always at least one moment where both old and new frontends work against whatever backend is live:

1. **Expand, don't break, first.** Deploy the backend so `/api/v1/pricing` and `/api/v2/pricing` coexist —

add the new endpoint without removing the old one. This is the "expand" phase of an expand/contract migration.

2. **Deploy the frontend** that calls `/api/v2/pricing`. Old cached frontends still work (v1 still exists); new frontends work (v2 exists).
3. **Wait out the tail** of old-client traffic. This tail can be long — bound it deliberately: check analytics/logs for the last request to `/api/v1/pricing`, and give it a generous grace window (days, not minutes) that accounts for cached `index.html` edge cases, offline PWA users reactivating stale service workers, and users who simply haven't hit refresh.
4. **Only then contract** — remove `/api/v1/pricing` from the backend, once telemetry shows zero (or acceptably near-zero) traffic to it.

```
Backend:  [add v2, keep v1] ----- (grace window, monitor v1 traffic) -----  
[remove v1]  
Frontend:                [deploy calling v2]  
                        ^ safe any time after v2 exists, well before v1 is removed
```

Practically, this means the backend team's "remove old endpoint" step should never be scheduled in the same release as the frontend team's "switch to new endpoint" step — they should be two separate releases, ordered as above, with an explicit monitoring gate between steps 3 and 4 (e.g., a dashboard alert threshold: "v1 traffic < 0.1% of total for 48 hours" before contract is allowed).

For genuinely breaking changes where "expand" isn't possible (e.g., a fundamentally incompatible response shape, not just a new endpoint), consider API-level content negotiation/versioning headers (`Accept: application/vnd.myapp.v2+json`) so the same URL can serve both shapes during the transition, or gate the frontend's new behavior behind a feature flag (Scenario 6, Option B) that's flipped on only after the backend's expand step is confirmed live everywhere (all instances, all regions) — decoupling "frontend build deployed" from "frontend feature active" gives an extra safety margin independent of deploy-pipeline timing.

Interviewer intent: Tests whether the candidate thinks beyond "did my deploy succeed" to the systemic reality of independently-deployed, differently-cached frontend/backend pairs, and knows the expand/contract migration pattern as the standard mitigation.

Scenario 15: Environment Variables Leaking Secrets Into the Client Bundle

Situation: A security audit finds a Stripe secret key and an internal admin API token hardcoded inside `environment.prod.ts`, which — because it's compiled into the client-side JS bundle — means any user could open browser DevTools, view the bundle source, and extract these secrets. This shipped to production for three months before being caught.

Question: Explain why this class of leak happens with Angular specifically, and how do you prevent it structurally (not just "remember not to do that") going forward?

Answer: Angular's `environment.ts` files are a **compile-time** mechanism — `ng build` inlines whatever is in the selected environment file directly into the output JS via `fileReplacements`. There is no server executing that code to keep secrets server-side; the entire bundle, including anything referenced from `environment`, is shipped verbatim to every browser and is trivially readable (even minified/tree-shaken code is just string-searchable — `strings dist/main.js | grep sk_live` will find a Stripe secret key in seconds).

The structural fix is a hard rule enforced by process, not just discipline: **anything secret (API keys with write/privileged scope, database credentials, admin tokens) must never be referenced anywhere in frontend source, including `environment.ts` — full stop.** Only genuinely public, non-sensitive values belong there (a Stripe *publishable* key, a public Mapbox token restricted by domain, a GA measurement ID — things explicitly designed to be exposed client-side).

Concretely:

1. **Move any operation needing a secret behind a backend endpoint.** If the frontend needs "create a Stripe payment intent," it calls `POST /api/payments/intent` on your own backend (which holds the secret key server-side and never returns it), rather than calling Stripe directly with a secret key from the frontend.
2. **Automate detection**, don't rely on code review alone — add a secret-scanning step to CI (`gitLeaks`, `truffleHog`, or GitHub's built-in secret scanning) that fails the build if a high-entropy string or known key pattern (`sk_live_`, AWS `AKIA...`, etc.) is detected anywhere in the repo, including environment files:

```
# CI pipeline step
- name: Scan for secrets
  run: |
    docker run --rm -v $(pwd):/repo zricethezav/gitleaks:latest detect --source /repo --
    exit-code 1
```

3. **Post-build bundle scanning** as a second safety net, since a secret could theoretically be introduced via a dependency, not just first-party code — grep the actual built output for known key patterns before deploy:

```
if grep -rE "sk_live_[A-Za-z0-9]{20,}" dist/app/browser; then
  echo "Secret key found in build output - aborting deploy"; exit 1
fi
```

4. **Rotate immediately** any secret confirmed to have shipped, since it must be treated as fully compromised the moment it left the build server — three months of exposure here means assuming it was harvested and used, not just "got lucky."
5. Educate the team on the specific mental model: **`environment.ts` is public.** It is exactly as exposed as a value hardcoded in `index.html` — there is no meaningful difference in exposure between "in `environment.ts`" and "in a comment in `index.html`," which is often the framing that finally makes this click for developers who intuitively (and wrongly) treat TypeScript source as "server-side-ish" because it doesn't look like a client-facing template.

Interviewer intent: A senior-level security-adjacent deployment question testing whether the candidate treats "the entire Angular bundle is public" as an absolute, load-bearing fact that shapes what's allowed in build-time config at all — and whether they propose durable, automated prevention (CI scanning) rather than just "be careful."

Scenario 16: Preview Environments for Every Pull Request

Situation: The team wants every open pull request to automatically get its own live, shareable preview URL (e.g., `pr-482.preview.example.com`) so designers/PMs can review UI changes without pulling the branch locally, and wants these environments torn down automatically when the PR closes.

Question: Design the CI/CD pipeline and hosting approach for ephemeral per-PR preview deployments of the Angular app, including how it should call a (shared, stable) backend/API.

Answer: This is a good test of applying the "one build, runtime config" and "host-based routing" principles (Scenarios 1 and 11) to an ephemeral, high-churn context, plus lifecycle management (creation and teardown tied to PR events).

Pipeline (GitHub Actions example):

```
name: PR Preview
on:
  pull_request:
    types: [opened, synchronize, reopened, closed]

jobs:
  deploy-preview:
    if: github.event.action != 'closed'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npx ng build --configuration=preview
      - name: Deploy to preview bucket path
        run: |
          aws s3 sync dist/app/browser s3://my-app-previews/pr-${{ github.event.number }} --
delete
      - name: Comment preview URL on PR
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              owner: context.repo.owner, repo: context.repo.repo,
              issue_number: context.issue.number,
              body: `Preview: https://pr-${context.issue.number}.preview.example.com`
            });

  teardown-preview:
    if: github.event.action == 'closed'
    runs-on: ubuntu-latest
    steps:
      - name: Remove preview artifacts
        run: aws s3 rm s3://my-app-previews/pr-${{ github.event.number }} --recursive
```

Routing: rather than provisioning a new CDN distribution per PR (slow, hits account limits fast), use one wildcard CDN distribution/Nginx config with a path- or hostname-derived rule (same technique as Scenario 11's multi-tenant routing) that maps `pr-482.preview.example.com` → `s3://my-app-previews/pr-482/`:

```
server {
    listen 443 ssl;
    server_name ~^pr-(?<prnum>\d+)\.preview\.example\.com$;
    ssl_certificate      /etc/ssl/wildcard-preview.example.com.crt;
    ssl_certificate_key  /etc/ssl/wildcard-preview.example.com.key;

    location / {
        proxy_pass https://my-app-previews.s3.amazonaws.com/pr-$prnum/;
    }
}
```

Backend/API strategy: almost always, preview frontends should point at a **shared staging API**, not a per-PR backend (spinning up a full backend+DB per PR is usually excessive unless the PR itself changes backend contracts, in which case a more advanced "ephemeral full-stack environment," e.g., via a preview-environments platform or a Kubernetes namespace-per-PR pattern, is justified but meaningfully more expensive/complex). Point the preview build's runtime config (per Scenario 1) at the shared staging API:

```
// config.preview.json baked in at preview-deploy time
{ "apiBaseUrl": "https://api-staging.example.com" }
```

Guard against real risk here: since preview environments are often unauthenticated/publicly reachable for reviewer convenience, make sure they never point at *production* data/API, and consider basic-auth-gating the preview hostnames (`auth_basic` in Nginx, or a signed-cookie check) if the app/company has any confidentiality requirement, since a predictable `pr-<number>.preview.example.com` URL pattern is easily enumerable by anyone.

Cache headers on preview deploys should generally be short/no-cache across the board (not just `index.html`) since preview builds churn constantly and long-lived CDN caching of a stale preview actively undermines the feature's purpose.

Interviewer intent: Tests whether the candidate can design a full ephemeral-environment lifecycle (create on open/push, teardown on close) rather than just a one-off deploy, and whether they correctly scope what's ephemeral (frontend) versus what should stay shared (backend/API, most of the time).

Scenario 17: Health Checks and Readiness Probes for a Containerized SPA

Situation: The Angular SPA is now served via an Nginx container inside Kubernetes (per the decision in Scenario 13). During a rolling deployment, Kubernetes starts routing traffic to new pods before Nginx inside them has actually finished starting, causing a burst of connection-refused errors for a few seconds on every deploy.

Question: Configure Kubernetes liveness/readiness probes correctly for this Nginx-serving-static-Angular-files pod, and explain the difference between the two probe types in this context.

Answer: Even though there's no application logic here (Nginx serving static files is about as simple a workload as exists), the rolling-update traffic-cutover problem is entirely a matter of correctly configured **readiness** vs **liveness** probes, which are commonly conflated.

- **Readiness probe** answers "should this pod currently receive traffic?" Kubernetes removes a not-ready

pod from the Service's endpoint list, so no traffic is routed to it, but does not restart it.

- **Liveness probe** answers "is this pod healthy enough to keep running at all?" A failing liveness probe causes Kubernetes to kill and restart the container.

The connection-refused burst described happens because the Service was sending traffic to a pod whose Nginx process hadn't finished binding to its port yet (or, more subtly, whose container was `Running` per Kubernetes but not yet actually accepting connections) — i.e., there was no readiness probe (or too permissive one) gating traffic.

```
# deployment.yaml (excerpt)
spec:
  template:
    spec:
      containers:
        - name: frontend
          image: registry.example.com/frontend:1.42.0
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 2
            periodSeconds: 5
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 15
            failureThreshold: 3
      # ensure rolling updates wait for new pods to be ready before killing old ones
    strategy:
      type: RollingUpdate
      rollingUpdate:
        maxUnavailable: 0
        maxSurge: 1
```

```
# nginx.conf - a dedicated lightweight health endpoint, not index.html itself
location = /healthz {
    access_log off;
    return 200 "ok\n";
}
```

Notes worth calling out:

- Using `/healthz` (a trivial static 200) rather than `/index.html` as the probe target avoids conflating "Nginx process is up" with "the app's actual content is being served correctly" in a way that's harder to reason about, and keeps probes cheap and fast (important since they run frequently).
- `maxUnavailable: 0` on the rolling update strategy ensures Kubernetes never takes an old, working pod

out of rotation until a new pod has passed its readiness probe and is confirmed able to serve traffic — this, combined with the readiness probe itself, is what actually eliminates the traffic-drop window, not the liveness probe (liveness only helps recover a wedged pod after the fact; it does nothing to prevent premature traffic routing to a not-yet-ready one).

- `initialDelaySeconds` on liveness should be comfortably longer than the readiness probe's, so a slow-starting-but-fine container isn't killed by liveness before it even had a chance to pass readiness.

Interviewer intent: Tests precise understanding of the readiness-vs-liveness distinction in a concrete rolling-deploy failure scenario — a frequently-confused pair of concepts even among engineers who've used Kubernetes for a while.

Scenario 18: Long TTL CDN Caching Serves the Wrong Region's Content After Failover

Situation: The Angular app is deployed to two AWS regions for disaster recovery (active-passive), fronted by a single CloudFront distribution with an origin group (primary + failover origin). During a regional outage, CloudFront correctly fails over to the secondary region's origin — but users continue seeing 5xx errors for several more minutes because CloudFront had cached the primary origin's 5xx *error responses* themselves.

Question: What CloudFront/CDN configuration mistake causes cached errors to persist through a failover, and how do you configure error caching and origin failover correctly?

Answer: By default, many CDNs (including CloudFront) will cache error responses (4xx/5xx) for a default TTL if the origin doesn't send explicit no-cache headers on the error response itself — and a struggling/dying origin very often does *not* reliably attach correct `Cache-Control` headers on its error pages (a 502/504 from an overloaded or dying origin frequently has minimal or default headers). If CloudFront cached that 5xx with even a short default TTL (CloudFront's default minimum error-caching TTL is 10 seconds unless configured, but misconfigurations often leave it much higher, or downstream caches/browsers add their own), requests continue being served the stale cached error even after failover succeeds and the secondary origin is healthy — because CloudFront's cache doesn't know to prefer "the origin group failover succeeded" over "I have a cached response for this exact request."

The fix has two parts:

1. **Explicitly minimize CloudFront's error-caching TTL** so any cached error is very short-lived, ensuring quick self-correction:

```
{
  "CustomErrorResponses": {
    "Items": [
      { "ErrorCode": 500, "ErrorCachingMinTTL": 0 },
      { "ErrorCode": 502, "ErrorCachingMinTTL": 0 },
      { "ErrorCode": 503, "ErrorCachingMinTTL": 0 },
      { "ErrorCode": 504, "ErrorCachingMinTTL": 0 }
    ]
  }
}
```

2. **Confirm the origin-group failover criteria include the actual error codes being seen** — CloudFront origin groups only fail over to the secondary origin for status codes explicitly listed in the failover

criteria; if 502/503/504 aren't all included, CloudFront may just keep hammering (and caching errors from) the primary even though it's down:

```
{
  "OriginGroups": {
    "Items": [{
      "FailoverCriteria": { "StatusCodes": { "Items": [500, 502, 503, 504], "Quantity": 4 }
    },
    "Members": { "Items": [{ "OriginId": "primary-region" }, { "OriginId": "secondary-region" }] }
  }
}
```

3. **Ensure the origin (Nginx or whatever serves `index.html`) sends `Cache-Control: no-store` on any error response it does manage to produce** (a 500 from an app-level failure, not just origin-unreachable) so downstream caches never treat a legitimately erroring response as cacheable content in the first place — pairing this with fix #1 covers both "origin sends bad/no cache headers on its error" and "CDN default behavior caches errors regardless."

The broader lesson: **caching policy must be reasoned about for error responses specifically, not just success responses** — the caching strategy from Scenario 4 (immutable hashed assets, no-cache `index.html`) is entirely about 200 OK responses; a parallel, deliberately short/zero error-caching policy is a separate, easy-to-forget configuration surface, and it's precisely the one that matters most during an actual incident/failover, which is exactly when you can least afford stale cached errors compounding the outage.

Interviewer intent: Tests whether the candidate thinks about CDN caching holistically (including error responses, not just asset caching) and understands CloudFront origin-group failover mechanics — a scenario that specifically catches people who've only ever reasoned about caching happy-path 200 responses.

Scenario 19: Angular Build Fails in CI Only, "Works on My Machine"

Situation: `ng build` succeeds locally for every developer but fails intermittently in the CI pipeline with out-of-memory errors (`JavaScript heap out of memory`) or, on a different day, with a dependency resolution error that doesn't reproduce locally. The team suspects the CI runner but can't pin down why.

Question: What are the systematic causes of "builds locally, fails in CI" for an Angular application-builder project, and how do you make the CI build environment reliably match (or at least reliably reproduce) developer machines?

Answer: This is really a build-reproducibility/deployment-pipeline-hygiene question, and there are several concrete, checkable causes rather than a vague "CI is flaky":

1. **Memory ceiling.** CI runners (especially shared/free-tier ones) often have materially less RAM than developer laptops (e.g., 4–7GB vs. 16–32GB). The esbuild-based application builder is faster and generally lighter than the old webpack builder, but large enterprise apps with many lazy chunks and heavy type-checking can still hit CI memory limits. Fix by explicitly bounding/raising Node's heap in the CI job (not infinitely — pick a value under the runner's actual limit) and/or splitting the build into fewer parallel concurrent jobs competing for the same memory:

```
build:
  script:
    - export NODE_OPTIONS="--max-old-space-size=4096"
    - npx ng build --configuration=production
```

Also confirm the CI job's memory *request/limit* (if containerized, e.g., in a Kubernetes-based CI runner) is actually large enough — a build silently OOM-killed by a container cgroup limit looks identical to a Node heap OOM from the log alone, but the fix (raise container memory limit vs. raise `--max-old-space-size`) is different.

- 2. Non-reproducible dependency resolution.** If the team uses `npm install` in CI instead of `npm ci`, or has `package-lock.json` out of sync/not committed, CI can resolve a different dependency tree than any given developer's local `node_modules` (which may have drifted from the lockfile over time via ad hoc local `npm install`s). `npm ci` strictly installs from the lockfile and fails fast if `package.json` / `package-lock.json` are inconsistent — always use it in CI:

```
- run: npm ci    # not npm install
```

- 3. Node/npm version drift.** Pin the exact Node version CI uses (`.nvmrc`, `engines` field enforced, or the CI config's explicit version) to match what's documented/enforced for local dev — Angular's supported Node version ranges change across major versions, and a CI runner silently using a newer/older default Node than developers can produce different esbuild/native-addon behavior.

```
// package.json
"engines": { "node": ">=20.11 <21" }
```

- 4. Case-sensitivity differences.** Developer machines on macOS/Windows have case-insensitive filesystems by default; CI runners (Linux) are case-sensitive. An import like `import { Foo } from './foo.service'` that actually resolves to `Foo.service.ts` on disk works locally but fails to resolve in CI — a very classic "works on my machine, fails in CI" cause specific to mixed-OS teams.
- 5. Environment-dependent code paths accidentally reached during build** (e.g., a build-time script that reads a local `.env` file present on developer machines but not committed/provided in CI) — ensure any build-time environment dependency is explicitly provided via CI secrets/variables, not assumed present.
- 6. Caching layer differences** — CI's `npm` / `node_modules` cache (if configured) can occasionally restore a corrupted or stale partial cache that differs from a clean local install; a CI-level "clear cache and retry once on failure" step, or simply not caching `node_modules` (only caching npm's download cache, and always running a full `npm ci` against it) avoids this class of intermittent failure.

The overarching deployment-pipeline principle: **CI should be treated as the single source of truth for "does this build," not developer machines** — which means investing in making CI's environment (Node version, memory, OS/filesystem semantics, lockfile-strict installs) a deliberately controlled, documented specification, rather than debugging drift after the fact each time it bites.

Interviewer intent: Tests CI/CD pipeline hygiene knowledge and whether the candidate can enumerate concrete, checkable causes (memory, lockfile drift, Node version, case-sensitivity) rather than shrugging at "flaky CI," which is a common real-world time sink.

Scenario 20: Zero-Downtime Deploy That Still Breaks In-Flight User Sessions Mid-Form

Situation: Despite implementing content-hashed assets, correct cache headers, and an atomic release-pointer flip (addressing Scenarios 2, 4, and 7), users still occasionally report losing form data / getting logged out / seeing a jarring reload in the middle of filling out a long multi-step form, specifically during deploy windows.

Question: Given that the "hard" caching/versioning problems are already solved, what's the remaining failure mode, and how do you make deploys genuinely transparent to a user with an in-progress session?

Answer: The remaining failure mode is that a "technically successful, no-stale-asset" deploy can still be *user-hostile* if the app forces a disruptive reload/logout the moment it detects a new version is available, which is exactly what a naive `swUpdate` /version-check integration (Scenario 9) or an aggressive "new version available, reloading in 3..." banner does. The infrastructure being correct doesn't mean the UX of *responding* to new-version-detection is correct.

Layered mitigations:

1. **Don't force-reload on version detection; ask, and only at a safe moment.** Track whether the user has unsaved state (a dirty form, an open modal, in-progress upload) and defer the reload prompt until the user reaches a natural checkpoint (form submitted, navigated away) rather than interrupting mid-edit:

```
export class AppUpdateService {
  private pendingUpdate = false;
  private formStateService = inject(FormDirtyTrackerService);

  constructor() {
    this.swUpdate.versionUpdates
      .pipe(filter((e): e is VersionReadyEvent => e.type === 'VERSION_READY'))
      .subscribe(() => {
        this.pendingUpdate = true;
        this.maybePromptForReload();
      });

    this.formStateService.dirtyStateChanged.subscribe(isDirty => {
      if (!isDirty && this.pendingUpdate) this.maybePromptForReload();
    });
  }

  private maybePromptForReload() {
    if (!this.formStateService.isDirty()) {
      this.showNonBlockingBanner('An update is available – refresh when convenient.');
```

2. **Persist form state client-side regardless**, as defense in depth against *any* unexpected reload (not just deploy-triggered ones — a browser crash or accidental tab close is just as disruptive to the user). Autosave to `localStorage` / IndexedDB on an interval or on blur, and restore on next load:

```
effect(() => {
  const value = this.form.getRawValue();
  localStorage.setItem('draft:onboarding-form', JSON.stringify(value));
});
```

3. **Session/auth token continuity across deploys.** If "getting logged out" is happening specifically at deploy time, check whether auth tokens are being invalidated by the deploy itself (e.g., a signing-key rotation shipped in the same release as an unrelated frontend change) rather than by anything reload-related — this is a backend-deploy coordination issue (see Scenario 14's expand/contract principle) masquerading as a frontend caching issue: rotate signing keys with a grace period where both old and new keys validate, never a hard cutover in the same release.
4. **Long-polling/websocket connections should reconnect gracefully,** not treat a backend restart during deploy as a fatal error requiring a full page reload — implement exponential-backoff reconnect logic in any persistent-connection service so a backend rolling-restart (completely normal and expected during a backend deploy) doesn't visually manifest as an error to the frontend user at all.
5. Set expectation with stakeholders: "zero-downtime deploy" at the infrastructure level (no failed HTTP requests, no 404s, no stale assets) is a different, necessary-but-insufficient guarantee from "zero-disruption deploy" at the UX level (no interrupted user workflows) — the latter requires this additional layer of application-level care (dirty-state-aware update prompts, autosave, graceful reconnect) on top of correct infrastructure, and should be called out explicitly as a separate work item rather than assumed to be automatically solved once caching/versioning/rollback are handled.

Interviewer intent: A senior-level capstone question checking whether the candidate can distinguish infrastructure-level "zero downtime" from user-experience-level "zero disruption," and whether they think about deploy-time UX (dirty forms, session continuity, reconnect logic) as a first-class deployment concern rather than assuming solved caching/versioning automatically implies a seamless user experience.

Quick Revision Cheat Sheet

- **Build once, configure at deploy time:** never use `fileReplacements` / `environment.ts` for per-environment URLs, feature flags, or tenant branding — fetch a runtime `config.json` (or use env vars for SSR) so the exact QA-tested artifact is what reaches production.
- **Hash everything the build controls, cache it forever:** `outputHashing: "all"` + `Cache-Control: public, max-age=31536000, immutable` on JS/CSS; never cache `index.html` (`no-cache, must-revalidate`) since it's the pointer to the current hashed set.
- **Retain old hashed assets for a grace period after deploy** so in-flight tabs referencing the old `index.html` don't hit `ChunkLoadError`; handle chunk-load failures client-side with a forced reload as a safety net.
- **SPA routing needs an explicit server-side fallback** (`try_files ... /index.html` in Nginx, `_redirects` / custom-error-response elsewhere) — deep links and refreshes are real HTTP requests, unlike in-app router-link navigation.
- `alias` vs `root` and `--base-href` matter the moment you host under a sub-path; get either wrong and both assets and deep links 404.
- **CORS errors in prod but not dev usually mean the dev-server proxy hid a same-origin decision**

you still must make explicitly — either reverse-proxy same-origin (preferred, kills CORS entirely) or configure real CORS with an explicit origin allow-list.

- **Blue-green and canary both need an atomic traffic-switch mechanism** (CDN origin-path flip, weighted routing, feature flag) and must keep the outgoing version's assets alive during the cutover window; canary at the CDN/infra level for cross-major-version rollouts, canary via feature flags for same-build incremental rollouts.
- **Rollback speed is designed in advance, not improvised:** immutable versioned releases + one atomic "what's live" pointer = a single command/click rollback with no rebuild; a feature flag flip can be even faster than a redeploy.
- **SSR ships two output trees** (`browser/` + `server/`); containers want `minInstances >= 1` to avoid cold-start-hurts-FCP irony, serverless trades that for pay-per-use at the cost of cold starts unless warmed.
- **Service workers deliberately don't self-update an open tab** — always wire `swUpdate.versionUpdates` and `checkForUpdate()`, and decide deliberately between prompting and force-reloading, ideally deferred to a safe (non-dirty-form) moment.
- **Multi-tenant/PR-preview hosting both reduce to the same pattern:** one build, host-based (or path-based) routing at the CDN/reverse-proxy layer, per-tenant/per-PR config resolved at runtime — never per-tenant or per-PR rebuilds.
- **Decoupled frontend/backend deploys need expand/contract sequencing:** add new API surface before the frontend depends on it, remove old API surface only after telemetry shows the old-client tail has died out — never treat frontend and backend deploys as atomic.
- **Kubernetes readiness (traffic gating) vs. liveness (restart-if-wedged) are different jobs** — a rolling deploy's traffic-drop problem is fixed by readiness probes + `maxUnavailable: 0`, not by liveness probes.
- **CDN error-response caching is a distinct, easy-to-forget policy surface** from success-response caching — set explicit low/zero error-caching TTLs and correct origin-group failover status codes so a regional failover isn't undone by cached 5xxs.
- **Anything in `environment.ts` (or any client bundle) is fully public** — secrets belong server-side behind an endpoint the frontend calls, never inlined into build-time config; enforce with automated CI/bundle secret scanning, not just code review.
- **"Zero downtime" (infrastructure) and "zero disruption" (user experience) are different guarantees** — dirty-form-aware update prompts, client-side autosave, and graceful reconnect logic are the additional application-level layer that makes deploys truly transparent to users mid-task.

Created By - Durgesh Singh